CampusCare: Digital Campus Healthcare Management System

Project Proposal for UCS503P
Submitted to: Ms. Anushka
Thapar Institute of Engineering and Technology

Lavdeep Singh, 1024030107 Harshveer Singh, 1024030109
Damanbir Singh, 1024030110 Kriti, 1024030111

August 19, 2026

1 Higher-order goal

This project aims to improve the efficiency, transparency, and continuity of healthcare services within a college campus by reducing unnecessary waiting, improving access to student medical history, and digitizing the consultation-to-dispensation workflow.

While the broader objective is to improve the campus healthcare experience, the immediate engineering goal is to translate the existing manual health-center workflow into a measurable and deployable software system.

The proposed system will connect student identity, health records, doctor-specific queues, consultation records, structured prescriptions, medicine dispensing, and administrative management within a single role-based platform.

The project focuses on workflow engineering rather than clinical decision-making. Doctor selection will remain under the control of the student and healthcare staff, while emergency cases will be handled separately from the routine consultation queue.

2 Time-to-value

- We will deliver meaningful increments quickly by implementing the smallest complete healthcare workflow first: student check-in → doctor queue → consultation → prescription → medicine dispensing → completed visit.
- We will prioritize a working end-to-end workflow over implementing every planned feature simultaneously.
- We will initially implement staff-assisted manual check-in so that the core healthcare workflow can be validated independently of NFC/RFID integration.
- NFC/RFID integration will be introduced after experimentally validating the compatibility of the actual college ID card, target Android device, browser environment, and deployment configuration.
- We will use short development iterations and continuous testing so that problems in the core workflow are identified before additional features are built on top of it.

- CI/CD will be introduced early to automatically build, test, and deploy changes to a staging environment.
- Success in the first iteration will be defined by whether a complete healthcare visit can be processed correctly rather than by the number of features implemented.

3 Problem Statement

The campus health center currently relies substantially on manual processes for student check-in, queue management, consultation records, prescriptions, and medicine dispensing. These processes create several operational problems:

- **Unclear queue status:** Students may have to wait without knowing their position in the queue or how many students are ahead of them.
- **Limited access to medical history:** Doctors may not have convenient access to a student's previous visits, prescriptions, or relevant medical conditions, resulting in consultations that may begin without sufficient historical context.
- **Paper-based prescriptions:** Handwritten prescriptions can be difficult to interpret, can be misplaced, and do not provide a structured digital record that can be conveniently accessed during future visits.
- **Limited dispensing records:** The relationship between what a doctor prescribed and what was actually handed to the student may not be captured in a centralized digital workflow.
- **Manual identity verification:** Students who do not have their college ID card require staff-assisted identification and check-in.
- **Fragmented information:** Student identity, health profile, consultation history, prescriptions, queue state, and dispensing information can exist across disconnected manual processes.
- **Limited administrative visibility:** Doctor availability, staff accounts, RFID/card enrollment, and other operational information require centralized management.
- **Different routine and emergency workflows:** Routine consultations and emergency/trauma cases require different handling and should not necessarily be forced through the same queue.

Why this matters now (contextual relevance): A campus health center handles a recurring flow of students with limited medical and administrative staff. Improvements in queue transparency, record accessibility, and workflow coordination can reduce avoidable waiting and repeated administrative work while improving continuity of student care.

4 Proposed Solution

4.1 Overview

Build a **web-based Progressive Web Application (PWA)** that digitizes the routine campus healthcare workflow while maintaining separate roles and responsibilities for students, doctors, pharmacists/receptionists, and administrators.

The proposed system will include:

- **Student portal:** students create and maintain their profile, provide relevant medical information, view their medical history and prescriptions, check in, select an available doctor, and monitor their queue status.
- **Identity-assisted check-in:** the existing college ID card will be investigated as an NFC/RFID-based check-in mechanism. Its identifier will be associated with the corresponding student account. Staff-assisted manual check-in will remain available when the card is unavailable.
- **Doctor-specific queues:** students join the queue of their selected doctor and can view their current queue status.
- **Doctor consultation workflow:** doctors call students, access relevant medical history, record consultation information, create structured prescriptions, and redirect students to another doctor's queue when necessary.
- **Structured prescription system:** prescriptions will use structured medication and dosage fields together with a general-instructions section rather than relying entirely on free-form handwritten-style text.
- **Pharmacist/receptionist workflow:** the authorized pharmacist/receptionist handles manual check-ins, verifies and records medicine dispensing, and closes completed visits.
- **Emergency recording:** emergency and trauma cases will be recorded through a separate workflow by the designated emergency staff member rather than being placed into the routine consultation queue.
- **Administrative dashboard:** administrators manage doctor and pharmacist/receptionist accounts, doctor availability, student-related administrative information, and RFID/card enrollment.
- **Role-based access control:** system functionality and medical information will be controlled according to the user's assigned role.
- **Audit logging:** important changes and actions can be recorded with the responsible user and timestamp for accountability.

4.2 Core workflow

1. A student authenticates into CampusCare and has an associated student profile.
2. At the health center, the student checks in using the college ID card if the NFC/RFID integration is available and has been validated. If the card is unavailable, the pharmacist/receptionist performs a staff-assisted manual check-in.
3. The student selects an available doctor based on the information provided by the system and joins that doctor's queue.
4. The student can view their current queue status through their CampusCare account.
5. The doctor calls the next student when ready.
6. The doctor views the student's relevant health profile and previous clinical records.
7. The doctor records the consultation and creates a structured prescription.
8. If the student has been directed to the wrong doctor, the doctor can redirect the existing visit to the appropriate doctor's queue without creating a duplicate visit.

9. After consultation, the visit enters a medicine-dispensing state.
10. The authorized pharmacist/receptionist verifies and records the medicines dispensed.
11. The original prescription remains the doctor's clinical record. Any difference between the prescribed and dispensed medicines is recorded separately rather than silently modifying the doctor's prescription.
12. Once dispensing is completed, the visit is closed and becomes part of the student's medical history.
13. Emergency and trauma cases bypass the routine queue and are recorded through the separate emergency workflow.

4.3 Operational constraints for an educational campus setting

- Keep the application usable on commonly available student and staff devices through responsive web design and PWA capabilities.
- Avoid dependence on advanced clinical decision-making or automated diagnosis; focus on workflow, access control, data integrity, and system correctness.
- Use common cloud-hosting and managed-database services so that the system can be deployed without requiring dedicated infrastructure.
- Keep NFC/RFID integration as a replaceable entry mechanism so that manual staff-assisted check-in remains available if hardware or browser limitations prevent NFC integration.

5 Solution Approach

This is an engineering course project, so the focus is on healthcare workflow, system correctness, database design, security, authorization, queue management, and deployability rather than novel clinical algorithm research.

5.1 Structured prescription and dispensing

The prescription system will use structured fields instead of storing the entire prescription as unrestricted free text.

Depending on the finalized prescription model, medication entries may contain:

- Medication name
- Strength
- Dosage amount
- Dosage unit
- Form
- Route
- Frequency or schedule
- Duration
- Before/after-food instructions where applicable

- As-needed instructions where applicable
- Additional instructions

The exact prescription structure will be finalized after reviewing established healthcare data models and determining which fields are appropriate for the campus health-center workflow.

The system will maintain a distinction between:

$$\text{Doctor's Prescription} \neq \text{Pharmacist's Dispensing Record}$$

The pharmacist/receptionist can therefore record what was actually dispensed without silently modifying the doctor's original clinical prescription.

The prescription structure may be informed by established healthcare interoperability concepts such as HL7 FHIR, without attempting to implement the complete FHIR standard.

5.2 Web architecture

A typical 3-tier web architecture will be used:

- **Frontend:** React + Vite + Tailwind CSS providing role-specific interfaces for students, doctors, pharmacists/receptionists, and administrators.
- **Backend API:** Python + FastAPI handling authentication, authorization, student profiles, health records, check-in, queue management, consultations, prescriptions, dispensing, emergency records, RFID enrollment, and administrative operations.
- **Database:** PostgreSQL storing users, student profiles, health information, RFID credentials, visits, queue entries, consultations, prescriptions, dispensing records, emergency cases, and audit information.

The database will use hand-written SQL and version-controlled migration files rather than relying on an ORM as the primary database abstraction.

Business rules and authorization will be enforced on the backend rather than relying only on frontend restrictions.

5.3 Authentication and authorization

CampusCare will use persistent accounts for students and authenticated accounts for staff.

The system will support four primary roles:

- **Student**
- **Doctor**
- **Pharmacist/Receptionist**
- **Administrator**

Students will be able to access their own medical information and visit history. Doctors, pharmacists/receptionists, and administrators will have access according to their assigned responsibilities and the finalized permission matrix.

The exact student authentication mechanism, including potential integration with the college's Google identity system, will be finalized after technical and institutional validation.

5.4 NFC/RFID identity integration

The existing college ID card will be investigated as the physical identity mechanism for student check-in.

The intended flow is:

$$\text{College ID Card} \rightarrow \text{Card Identifier} \rightarrow \text{Student Account} \rightarrow \text{CampusCare Visit}$$

The card's identifier will not itself be treated as the student's medical identity. Instead, it will act as an identifier mapped to the student's account.

The actual compatibility of the college card with the browser-based Web NFC approach will be experimentally verified before this mechanism is treated as a confirmed system dependency.

5.5 Queue management

Each doctor will maintain a separate queue.

The queue system will support:

- Student joining a doctor's queue
- Viewing queue status
- Doctor calling the next student
- No-show handling
- Doctor redirection
- Queue completion
- Doctor availability

If a student does not appear after being called, the system will use a defined buffer period. After the buffer expires, the student can be moved to the end of the relevant queue and the doctor can proceed with the next student.

The exact no-show duration will be finalized after observing the actual health-center workflow.

5.6 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build the frontend and validate the backend on every change.
- Run unit and integration tests.
- Perform linting and other automated checks.
- Produce repeatable deployment artifacts.
- Deploy successful builds to a staging environment.
- Reduce integration problems between the four team members by continuously validating changes.

6 Evaluation Criterion

We define success using measurable metrics tied to the core healthcare workflow.

6.1 Primary evaluation metric

Check-in-to-queue-entry time: time between the beginning of a student's check-in process and successful creation of the student's queue entry.

The system will record timestamps for:

$$\text{Check-in Initiated} \rightarrow \text{Identity Verified} \rightarrow \text{Doctor Selected} \rightarrow \text{Queue Entry Created}$$

The exact target value will not be assumed in advance. An initial baseline will be measured from the existing manual workflow before defining a target improvement.

- **Measurement:** database timestamps generated by the system.
- **Baseline:** observations or measurements from the existing manual workflow.
- **Attribution:** timestamps recorded by the CampusCare system rather than subjective estimates.

6.2 Secondary evaluation metrics

- **Queue transparency:** whether students can successfully determine their selected doctor, current queue status, and whether they have been called.
- **Waiting time:** time from successful queue entry to being called by the doctor.
- **Visit completion time:** time from check-in through consultation, dispensing, and final visit closure.
- **Record accessibility:** time or number of steps required for a doctor to retrieve a student's relevant previous medical records.
- **Prescription completeness:** percentage of prescriptions containing all required structured fields.
- **Dispensing accuracy:** percentage of visits where prescribed and dispensed medicines are correctly recorded, including explicit documentation of differences.
- **Reliability:** availability and successful operation of the deployed system during the defined pilot period.
- **Authorization correctness:** successful prevention of actions outside a user's assigned role.

6.3 Pilot validation plan

- Initially use synthetic student and medical data for development, testing, and demonstration.
- Use real student medical information only if the institution explicitly authorizes its use and the deployment environment satisfies the required privacy and security requirements.
- Recruit representative student, doctor, pharmacist/receptionist, and administrator test users for workflow evaluation.
- Execute representative workflows including check-in, doctor selection, queue monitoring, consultation, prescription creation, dispensing, visit completion, doctor redirection, no-show handling, emergency recording, and administrative operations.
- Compare system-generated metrics with observations of the existing manual workflow.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in the number of students, visits, doctors, medical records, and concurrent sessions by:
    - separating frontend and backend components,
    - using pagination for large result sets,
    - using indexes for common database queries,
    - maintaining normalized relational data,
    - using transactions for queue and visit state changes,
    - using stateless API design where practical.
2. **Operationally deployable (practically):** The system should run using common web hosting and managed database infrastructure:
    - managed PostgreSQL database,
    - platform-hosted frontend and backend,
    - environment-based configuration,
    - CI/CD pipeline for staging and production deployment.

8 Engine availability heuristic

A mature set of software technologies and services is available to implement CampusCare as a web-based project:

- **React + Vite:** frontend application development.
- **Tailwind CSS:** responsive interface development.
- **FastAPI:** REST API and backend development.
- **PostgreSQL:** relational database management.
- **Raw SQL and migration tooling:** explicit database design and version-controlled schema evolution.
- **Web NFC:** proposed browser-based NFC integration, subject to validation using the actual college ID card and target Android device.
- **Authentication infrastructure:** persistent student and staff authentication.
- **Testing frameworks:** unit and integration testing.
- **CI/CD tooling:** automated validation and deployment.
- **Managed cloud hosting:** deployment of frontend and backend services.
- **Managed PostgreSQL:** cloud-hosted relational database for staging or production deployment.

We will use these mature components to focus on workflow design, correctness, security, database engineering, and measurement rather than building infrastructure from scratch.

9 Project scope and deliverables

9.1 Initial deliverable

The first iteration will implement the smallest complete healthcare workflow:

- Student account and authentication
- Student profile
- Staff-assisted manual check-in
- Doctor selection
- One doctor's queue
- Queue position/status
- Doctor authentication
- Doctor calls next student
- Doctor views relevant student medical history
- Doctor records consultation
- Doctor creates a structured prescription
- Pharmacist/receptionist views the prescription
- Pharmacist/receptionist records medicine dispensing
- Visit completion
- Completed visit appearing in student history
- PostgreSQL persistence
- Backend REST API
- Basic role-based authorization
- CI build and backend tests
- Deployment to a staging environment

The objective is to prove that a complete healthcare visit can move successfully through the system rather than maximizing the number of screens or optional features.

9.2 Subsequent deliverables

- Second doctor and independent doctor-specific queue.
- Doctor availability management.
- Doctor redirection between queues.
- No-show handling and queue reordering.
- Complete student medical history and prescription history.

- Improved clinical-history interface for doctors.
- More complete role-based permission rules.
- NFC/RFID check-in.
- RFID enrollment and revocation.
- Card-to-student identifier mapping.
- Emergency/trauma recording.
- Administrative dashboard.
- Pharmacist dispensing workflow improvements.
- Audit logging.
- Polling-based live queue updates.
- WebSocket-based queue updates if time permits.
- Additional testing and security hardening.

9.3 Stretch deliverables

Only after the complete core system is stable, the following features may be attempted:

- **Medication reminders:** reminders generated from structured prescription schedules.
- **Disease-surge indicator:** aggregated and anonymized visit-category information used to flag unusual increases in particular categories of illness for administrative decision support.

9.4 Explicitly out of scope

Medicine inventory management will not be implemented during the current semester.

Inventory management is considered a natural future extension because it can connect dispensing records with medicine stock, but it is intentionally excluded from the current scope to keep the core system achievable within the semester.

10 Risks and mitigations

- **NFC/Web NFC compatibility:** The actual college card may not expose the required identifier through the browser environment. We will perform an early feasibility test using the actual card, target Android device, browser, and HTTPS deployment before making NFC a hard dependency.
- **Medical data privacy:** Medical records are sensitive information. Development and demonstration will use synthetic data unless the institution explicitly authorizes real data use and the required privacy and security controls are in place.
- **Unauthorized medical-record access:** Different roles require different levels of access. Backend-enforced role-based authorization and a clearly defined permission matrix will be implemented before exposing medical records.
- **Prescription modification:** The pharmacist/receptionist must not silently alter the doctor's clinical prescription. Doctor prescriptions and dispensing records will therefore be stored separately.

- **Queue inconsistency:** Concurrent requests may create race conditions in queue operations. PostgreSQL transactions and appropriate concurrency controls will be used for queue state changes.
- **Scope creep:** NFC, WebSockets, reminders, analytics, emergency management, administration, and structured prescriptions could collectively exceed the semester timeline. Core functionality will therefore be completed before stretch features are attempted.
- **Student identity mismatch:** Student identity information should not rely entirely on unrestricted manual input. Institutional authentication or verified student information will be preferred where available.
- **No-show handling:** A student who does not appear after being called can block the doctor queue. A defined buffer period will be used, after which the student can be moved to the end of the relevant queue.
- **Doctor availability changes:** Changing doctor availability while students are already queued can create inconsistent states. Explicit rules for existing queues will be defined before availability management is implemented.
- **Deployment issues:** Cloud deployment or database configuration problems can disrupt demonstrations. Staging deployment will be introduced early and deployment configuration will be maintained through version-controlled project configuration and CI/CD.

11 Summary

This project proposes a deployable web platform to improve the organization and transparency of routine healthcare services within a college campus.

CampusCare connects student identity, optional NFC/RFID-assisted check-in, doctor-specific queues, digital medical history, structured prescriptions, pharmacist dispensing, and administrative management within a role-based architecture.

The project emphasizes **engineering workflow and system correctness rather than clinical automation**. Routine consultations follow a controlled workflow from check-in to queue, consultation, prescription, dispensing, and completion, while emergency and trauma cases are deliberately handled through a separate workflow.

The initial implementation will use a walking-skeleton strategy, beginning with manual check-in and one complete doctor workflow before progressively introducing the second doctor, queue management features, NFC/RFID integration, emergency recording, administrative functionality, and improved real-time updates.

The technical architecture uses React/Vite/Tailwind on the frontend, FastAPI for the backend, and PostgreSQL with raw SQL and version-controlled migrations. CI/CD, automated testing, role-based authorization, transaction-safe queue management, and audit logging will support incremental development.

The project will evaluate measurable aspects of the workflow, including check-in-to-queue-entry time, waiting time, queue transparency, visit completion, prescription completeness, dispensing accuracy, system reliability, and authorization correctness.

Medicine inventory management is intentionally excluded from the current semester, while medication reminders and anonymized disease-surge indicators remain optional stretch goals.

The project therefore prioritizes delivering a **complete, secure, measurable healthcare workflow first**, with additional functionality introduced only after the core system is stable.